A Strong Harness Narrows the Model Gap: A Comparative Evaluation for Legislative Document-to-XML Conversion

Palvika Bansal*, **Rittika Jindal***, **and Ruichong "Alex" Wang***
Thomson Reuters Technology
{palvika.bansal, rittika.jindal, alex.wang}@thomsonreuters.com

arXiv:2610.19999v1 [cs.CL] 14 Oct 2026

Abstract

Legal-content reproduction requires matching the source exactly in section hierarchy, verbatim text, citations, and amendment spans; a dropped citation or mislabeled amendment carries into the products lawyers rely on. We report an industrial pipeline converting U.S. state legislation into strict, schema-constrained XML, and hypothesize that its accuracy comes from harness engineering, the deterministic tooling built around a model, more than the model itself. Our controlled ablation crosses three harnesses (zero-shot, a single tool-using agent, and a decomposed multi-agent pipeline) with three Claude models on 153 gold bills from two jurisdictions. The harness narrows the model gap: the weakest model in the full pipeline beats the strongest model alone (0.948 versus 0.806, paired bootstrap p less than 0.001). Because the two agentic harnesses share identical tools, their gap comes from orchestration: decomposition plus the persisted state and per-agent context it brings. Across four scored dimensions, the model spread at zero-shot (0.68 to 0.81) narrows to 0.95 at the full pipeline; the residual sits almost entirely in citation normalization, where the canonical identifier is retrieved from an external authority rather than generated from the weights, so a deterministic lookup tool supplies the exactness that a larger model does not.

1 Introduction

Most legal-NLP benchmarks reward understanding. Classification, retrieval, judgment prediction, and summarization all ask whether a system recovers the meaning or the right label, and a paraphrase that keeps the substance usually counts as correct. Reproduction is a different problem. Once a bill becomes a product that lawyers rely on, the output has to match the source down to the surface: sections and subsections in the right order, the bill text verbatim, citations in canonical form, the struck and inserted spans of an amendment placed where they belong, and the metadata that indexes it downstream. A dropped citation or a mislabeled amendment is an error that travels into the products lawyers put in front of clients, wherever in the document it happens to sit. What we are after is exact reproduction, not a high average.

A large enough model might be expected to clear this bar on its own. It does not. In the zero-shot condition (T1), even the strongest model we tested falls short: Opus reaches only 0.806 overall and barely resolves jurisdiction-specific citations (normalization 0.232). The shortfall is not obviously something a larger model fixes. A model has no way to know a jurisdiction's normalized statutory identifiers from pre-training, since that mapping lives in an external authority rather than in the weights, and making the model larger does little to close the gap.

Our working hypothesis is that exactness here owes more to harness engineering, the orchestration, tools, and deterministic constraints assembled around the model, than to the model itself. To test it we cross three harnesses with three models (§4): T1 runs the model zero-shot with no harness, T2 hands a single agent the full tool set and asks it to self-sequence the conversion, and T3 is the decomposed harness modeled on the production system, splitting that same tool set across specialized agents. Because T2 and T3 share tools exactly, whatever separates them should reflect orchestration—decomposition together with the persisted state and per-agent context it brings—not a wider action space.

We make four contributions. **(1) A deployed harness**: a playbook-driven, multi-agent pipeline that converts U.S. state legislation into schema-constrained publication XML and runs in production at a legal-content publisher (§4). **(2) A controlled model×harness ablation** that holds tool

* Equal contribution.

access constant between the single-agent (T2) and decomposed (T3) harnesses, which lets us attribute the gain to orchestration—decomposition, the persisted state it introduces, and per-agent context isolation—rather than to a larger action space (§6). **(3) A dimension-resolved error analysis** showing the residual error is not spread evenly but settles in the legally load-bearing dimensions, citations and amendments, and on the harder jurisdiction (§6). Handing the citation-lookup tool to a single agent at T2 does not appear to be enough on its own: only the stronger models invoke it well (normalization 0.88 and 0.59 for Sonnet and Opus against Haiku's 0.10), and the weakest recovers only once T3 gives it a dedicated citation agent (0.80, §6.3). **(4) An evaluation built for legal-document transformation**: we decompose correctness into four dimensions, pairing standard document measures (character-level normalized edit distance (NED) and tree-edit similarity) with two domain measures, exact amendment-span matching and a citation score split into localization and normalization. That split is what localizes the residual to citation normalization, which no model in our study resolves unaided (§5).

2 Related Work

Long-document structure and schema generation. Systems that convert raw documents into structured markup, such as the OCR-free parsers Nougat (Blecher et al., 2024) and GOT-OCR (Wei et al., 2024), are judged with the fidelity metrics we also use: normalized edit distance (Levenshtein, 1966) and Tree-Edit-Distance Similarity (Zhong et al., 2020), codified by OmniDocBench (Ouyang et al., 2025). A closer benchmark, ParseBench (Zhang et al., 2026a), finds no method consistently strong across dimensions, so general-purpose parsing is not yet solved even before any legal requirement enters. None of this work, though, scores against a schema with legal-semantic markup such as amendment spans and resolved citations, at our largest documents' scale (§3), or as a function of system architecture. Our schema also differs from the legislative-markup standards USLM[1] and Akoma Ntoso[2] (the OASIS LegalDocML standard), since we evaluate against a fixed, confidential in-house schema. Grammar-constrained decoding (Geng et al., 2023) is related but separate: it guarantees well-formed output at inference time, which is not the same as measuring a full pipeline's fidelity once assembled.

Agentic harnesses and evaluation. Our T3 pipeline draws on now-standard agentic patterns: reasoning interleaved with tool calls (Yao et al., 2023; Schick et al., 2023) and role-specialized agents (Wu et al., 2024; Hong et al., 2024). Benchmarks for such systems (Liu et al., 2024; Jimenez et al., 2024; Zhou et al., 2024) tend to score task completion and usually vary tool access and decomposition together, so a gain from adding agents is hard to separate from adding tools. A more recent line names the surrounding infrastructure the *harness* and argues it, not the base model, often decides whether a capability becomes a finished task. Harness-Bench (Yao et al., 2026) reports capability at the model×harness level, while Zhang et al. (2026b) argue comparisons stay uninterpretable unless the harness is disclosed and held fixed. We add a domain instance under conditions that line has not jointly imposed. We grade against a hard exactness bar rather than completion, so the claim that the harness matters is tested where being almost right already counts as failure. And our T2-versus-T3 contrast is the fixed-harness comparison Zhang et al. (2026b) ask for, tools held identical while decomposition varies, which lets the gain read as orchestration rather than a larger action space.

Legal informatics. Applied legal NLP tends to grade a coarser correctness. Benchmarks like LegalBench (Guha et al., 2023) and CUAD (Hendrycks et al., 2021) score classification, extraction, and QA on label accuracy or span F_1, tolerating a near-miss that a legal-publishing task cannot: a clause classifier that mislabels one clause in twenty is still useful, but a bill-compilation system that drops one amendment span in twenty misstates an intended change to the law. Work on statutory citation processing (Darji et al., 2026) has treated resolution as a standalone extraction task. We measure it instead as one dimension of a full pipeline, split into localization and normalization (§5). Exact reproduction of a structured legal document remains comparatively understudied relative to the effort it absorbs.

[1] https://github.com/usgpo/uslm

[2] https://www.oasis-open.org/committees/tc_home.php?wg_abbrev=legaldocml

3 Task and Dataset

The task is to convert a legislative bill PDF into structured XML that reproduces the bill exactly, down to its section hierarchy, verbatim text, statutory citations, and amendment spans, and that validates against a fixed publication schema.

Source material. Our inputs are the legislative PDFs that state legislatures publish, here from the Idaho Legislature[3] and the New Jersey Legislature.[4] They bring all the difficulty of real legal typography: running headers and footers, margins, multi-column layouts, and footnotes. Amendatory bills add inline formatting on top of this, underline for inserted material and strikethrough for deleted material, which has to be recovered exactly rather than approximately. Text frequently spills across page and column boundaries, so getting the reading order right means reassembling content across physical breaks. Appendix A works a real public Idaho bill through the full PDF→XML pipeline as an example.

Target schema. The output has to validate against a strict publication XML schema constrained by a DTD (Document Type Definition), which we keep confidential. It encodes the section hierarchy (title, chapter, section, subsection, with typed paragraph containers), the verbatim bill text, citations resolved to existing statutory identifiers, amendment markup as typed inline elements for added and deleted material, and metadata for indexing. Validity is all-or-nothing, since a single misplaced element can invalidate the whole document. Appendix A shows this amendment markup worked through on a real bill.

Dataset. The gold set holds 153 expert-produced documents, 103 from Idaho and 50 from New Jersey, spanning the bill revision lifecycle (as-introduced, engrossed, adopted, and amended-substituted versions). Sizes range widely but skew small. The median document has about 130 body nodes (roughly 5.8K characters) and a nearly constant tree depth of 8, since the schema hierarchy is shallow. The tail runs long, out to two New Jersey budget bills large enough to strain the evaluation itself: SB 2025 at 40,944 structural nodes and AB 4700 at 8,229,[5] both past the node cap of the tree-edit-similarity scorer (§5). Appendix Table 5 gives the full size distribution by jurisdiction. The two jurisdictions differ in kind. Idaho is a fairly stable baseline, whereas New Jersey is harder across the board, with larger bills, heavier revision lifecycles, and more involved citation patterns. That matters, because New Jersey is where the harness's advantage over the model turns out to be widest (§6).

[3] https://legislature.idaho.gov/sessioninfo/

[4] https://www.njleg.state.nj.us/bill-search

[5] Both are New Jersey budget bills; see https://www.njleg.state.nj.us/bill-search/2024/S2025 (SB 2025) and https://www.njleg.state.nj.us/bill-search/2024/A4700 (AB 4700) for bill text.

4 Harness Engineering: Three Levels of Scaffolding

By harness we mean the deterministic engineering built around the model: typed tools, schema validation, per-jurisdiction configuration, and structural boundaries that fix what the model produces at each step. The three harnesses add this scaffolding in increasing amounts, none in T1, tools in T2, tools plus decomposition in T3 (Figure 1), and the paper's question is how much exactness each layer contributes.

T1: no harness, the zero-shot baseline. T1 is the model with no harness at all: we give it the source document in one pass and ask for the publication XML directly, with no tools, retrieval, or intermediate steps. This is the floor the other harnesses build on. Because it has no legal-code lookup, T1 also cannot resolve a citation to its canonical identifier, a limitation that dominates the results.

T2: a monolithic harness, the single-agent baseline. T2 adds tools but not decomposition, a harness, but an undifferentiated one. It gives one agent the complete tool set, the same 47 tools the production pipeline uses, and asks that single agent to sequence the whole conversion itself (extract, structure, convert, resolve citations, validate) in one context with no sub-agents. T2 is the pivotal control: it shares the model, tools, and data with T3 at constant tool access, so the T2→T3 gap isolates *orchestration* rather than a larger action space. We use that term for three coupled changes T3 introduces together: decomposition into specialized agents, the persisted checkpointed state that decomposition makes possible, and the smaller separate context each specialist works in, whereas T2 threads everything through one long context.

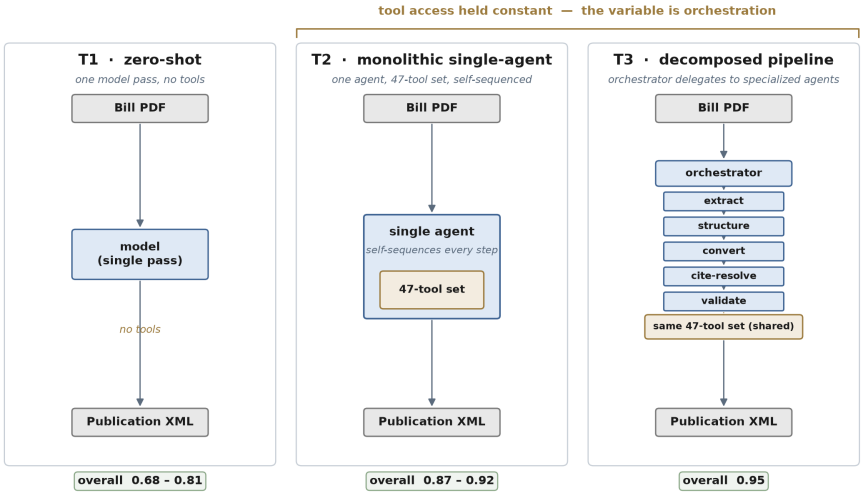


Figure 1: The three harnesses, all starting from the same bill PDF and ending at the same publication XML. Because tool access is held constant between T2 and T3, the difference between them isolates orchestration: decomposition, the persisted state it enables, and the per-agent context it creates.

T3: a decomposed harness, modeled on the production pipeline. T3 mirrors the deployed system's architecture, with one simplification for experimental control: every agent runs the same model, whereas the deployed system mixes a stronger orchestrator with lighter task agents (see the Limitations section). It separates workflow definition from execution. An editable natural-language playbook, which subject-matter experts change without touching code, is read by a generic workflow agent that hands each step to a specialized task agent: document extraction, structure and auxiliary-artifact construction, statute-impact extraction, XML conversion, citation resolution and footnoting, validation. Each agent therefore works on a narrow, well-typed subproblem with a small context and action set. The tool set is identical to T2's; only the orchestration differs. Both harnesses carry state between steps (extracted structure, intermediate XML, resource and revision identifiers, metadata), and how each handles it is where the weaker model becomes unstable in T2 (§6). Appendix C gives the full pipeline and the T2-vs-T3 state-handling contrast.

What each harness is given. Table 1 sets out the inputs. All three tiers receive the bill as a native PDF, the same target-format XML templates, and the same per-jurisdiction conversion rules; T1 carries these in a single hand-authored instruction, while T2 and T3 load them at run time. The rule we held to is that only the harness may vary across tiers, never the knowledge, so a capability T2 and

	T1	T2	T3
Bill PDF (native)	yes	yes	yes
XML templates	yes	yes	yes
Jurisdiction rules	yes	yes	yes
Step-ordered playbook	none	9 steps	21 steps
Tools	none	47	17
Sub-agents	none	none	7
Persisted state	none	none	yes

Table 1: What each harness receives. Rows above the middle rule are *knowledge* and are held constant; rows below it are *harness*. T2 and T3 reach the same underlying tools: T2 calls the 47 leaf tools directly, while T3 reaches those same 47 leaves through 17 composite orchestrator tools—the counts differ in exposure, not in access. The step counts differ because T2's playbook stops at the scored artifact, while T3's continues into the publication stages that follow it.

T3 reach through a tool is written out in prose for T1 rather than withheld. What T1 cannot be given this way is anything requiring an external lookup or an intermediate artifact. T2 additionally receives a nine-step playbook covering the same conversion work T3's playbook describes, so the step ordering and per-step guidance are not what separates them.

Tool ecosystem. Agents reach backend capability over the Model Context Protocol (MCP). One structural property of it matters for the ablation: MCP is a typed request/response interface with no shared memory across the boundary. That is exactly what lets us hand the *same* tool set, unchanged, to both T2 and T3, the invariant the T2-vs-T3 comparison rests on. Six MCP-backed services back the